

RELATÓRIO FINAL – IMPLEMENTAÇÃO DE TRANSFERÊNCIA CONFIÁVEL DE DADOS E TCP SOBRE UDP

Integrantes:

- Enzo de Almeida - 23001396
- Rafael Celeste - 23007553
- Tiago Mello - 23014024
- Thomas Fidelis - 23013133
- Vinicius Nunes - 23011585

Campinas – 2025

1. Introdução

A confiabilidade na transmissão de dados é um dos pilares fundamentais das comunicações em redes de computadores. Em ambientes reais, as mensagens trafegam por meios sujeitos a perdas, atrasos e corrupções, tornando necessário o uso de protocolos que assegurem a entrega correta e ordenada das informações. O presente projeto, intitulado “Implementação de Transferência Confiável de Dados e TCP sobre UDP”, tem como objetivo desenvolver e analisar versões progressivas desses mecanismos, simulando o funcionamento de protocolos confiáveis sobre um meio de transporte não confiável, o UDP (User Datagram Protocol).

O UDP, por natureza, não estabelece conexão nem garante que os pacotes cheguem ao destino, sendo amplamente utilizado em aplicações que priorizam velocidade em detrimento da confiabilidade, como transmissões de vídeo ou jogos online. Em contrapartida, o TCP (Transmission Control Protocol) implementa diversas estratégias de controle de erro, fluxo e congestionamento para garantir a entrega correta das mensagens. Assim, compreender os princípios que regem a confiabilidade do TCP, implementando-os manualmente sobre o UDP, permite explorar de forma prática conceitos cruciais da camada de transporte.

O trabalho foi desenvolvido em Python, empregando sockets UDP e um canal de comunicação não confiável, capaz de simular perda, corrupção e atraso de pacotes. Foram implementadas três fases principais: (i) RDT 2.0, 2.1 e 3.0, responsáveis pela transferência confiável ponto a ponto; (ii) Go-Back-N (GBN), que introduz o envio em janelas deslizantes; e (iii) TCP Simplificado, que será desenvolvido posteriormente. Cada etapa reflete a evolução de técnicas usadas na prática por protocolos reais, oferecendo uma base sólida para o entendimento dos mecanismos de confiabilidade e desempenho em redes.

Ao longo do relatório, serão descritas as etapas de desenvolvimento, os desafios encontrados, as análises de desempenho e as diferenças entre os protocolos simulados. O estudo busca, além de validar os resultados obtidos experimentalmente, relacionar a teoria de redes com a implementação prática, promovendo uma compreensão aprofundada sobre os princípios que permitem a comunicação confiável na Internet.

2. Fase 1 – Implementação dos Protocolos RDT (Reliable Data Transfer)

A primeira fase do projeto tem como objetivo implementar versões progressivas do protocolo Reliable Data Transfer (RDT), partindo de um modelo simples até alcançar um sistema tolerante a perdas e atrasos. Essa etapa busca compreender, por meio da prática, como se constrói a confiabilidade sobre um canal não confiável — simulando o comportamento de transmissão e recepção de pacotes via UDP.

2.1. Fundamentos do RDT

Os protocolos RDT são responsáveis por assegurar que os dados enviados de um emissor cheguem corretamente ao receptor, mesmo quando há falhas na comunicação. Para isso, utilizam-se mecanismos como confirmações (ACKs), rejeições (NAKs), números de sequência e temporizadores (timeouts). Cada versão do protocolo acrescenta novas funcionalidades que corrigem limitações das anteriores, refletindo a evolução gradual até o modelo de confiabilidade adotado pelo TCP.

O canal de comunicação utilizado nesta implementação foi simulado pelo módulo `simulator.py`, capaz de introduzir aleatoriamente perdas, corrupções e atrasos de pacotes, de modo a reproduzir o comportamento de uma rede real. As mensagens trafegam encapsuladas em pacotes criados pelo módulo `packet.py`, que define o cabeçalho com tipo (DATA, ACK ou NAK), número de sequência e checksum — um valor hash gerado sobre os dados, utilizado para detectar corrupção.

2.2. Protocolo RDT 2.0 – Detecção de Erros

O RDT 2.0 introduz o conceito de detecção de erros. A cada mensagem enviada, o emissor calcula um checksum, que é reenviado junto aos dados. O receptor, ao receber o pacote, recalcula o checksum e compara os valores. Caso haja divergência, o receptor identifica a corrupção e envia um NAK solicitando retransmissão. Caso contrário, um ACK confirma o recebimento correto.

A implementação utiliza sockets UDP para enviar e receber pacotes, com verificação de integridade realizada antes de qualquer confirmação. Apesar disso, o RDT 2.0 não trata o problema da perda de pacotes — caso um ACK ou NAK se perca, o emissor fica bloqueado indefinidamente. Isso limita seu uso em cenários de rede real.

Nos testes realizados (`test_fase1.py`), foi possível observar o funcionamento correto da detecção de erros, porém com vulnerabilidade a perdas. Essa limitação motivou o aprimoramento na versão seguinte.

2.3. Protocolo RDT 2.1 – Controle de Duplicidade

O RDT 2.1 soluciona as ambiguidades de retransmissão presentes no RDT 2.0 por meio da introdução de números de sequência alternados (0 e 1). Cada pacote e seu respectivo ACK/NAK possuem um número de sequência que permite ao emissor identificar se a resposta recebida corresponde ao pacote atual ou a uma retransmissão anterior.

Na prática, isso elimina a duplicação de mensagens e evita que ACKs atrasados causem inconsistências. O emissor alterna o número de sequência a cada mensagem enviada e mantém controle sobre o último ACK válido. Essa técnica é conhecida como stop-and-wait, pois o emissor aguarda a confirmação antes de prosseguir com o próximo envio.

Os experimentos mostraram que o RDT 2.1 é capaz de operar com taxas de corrupção de até 30%, garantindo entrega correta sem duplicação. Entretanto, ainda não lida com perda de pacotes — um avanço introduzido no RDT 3.0.

2.4. Protocolo RDT 3.0 – Tolerância a Perdas

O RDT 3.0 é considerado o primeiro protocolo completamente confiável, pois adiciona o mecanismo de timeout. Cada pacote enviado inicia um temporizador; caso o ACK correspondente não chegue dentro do tempo estipulado, o emissor assume que o pacote ou o ACK foi perdido e retransmite automaticamente a mensagem.

Esse comportamento aproxima o protocolo de um modelo de rede real, onde atrasos variáveis e perdas são comuns. O RDT 3.0, portanto, é capaz de lidar simultaneamente com corrupção, perda e duplicação, tornando-se o ponto de transição entre o modelo básico e os protocolos de janelas deslizantes (como o Go-Back-N).

Durante os testes, o canal foi configurado com 15% de perda e 0% de corrupção, com atrasos aleatórios entre 50 e 500 milissegundos. O protocolo apresentou retransmissões controladas e throughput estável, com eficiência média de 80%. Esse desempenho comprova a robustez do modelo e sua adequação para redes reais.

2.5. Conclusões Parciais da Fase 1

A Fase 1 permitiu compreender os princípios fundamentais de confiabilidade na camada de transporte. A evolução entre as versões demonstrou a importância de acrescentar gradualmente mecanismos de controle de erro e temporização.

Versão	Tolerância a Corrupção	Tolerância a Perda	Controle de Sequência	Timeout	Tipo de ACK
RDT 2.0	Sim	Não	Não	Não	ACK/NAK
RDT 2.1	Sim	Parcial	Sim (0/1)	Não	ACK/NAK
RDT 3.0	Sim	Sim	Sim (0/1)	Sim	ACK

Os resultados obtidos confirmaram que a inclusão do temporizador é essencial para atingir comunicação totalmente confiável. Essa etapa serve de base conceitual e prática para o desenvolvimento da Fase 2, onde o modelo evolui para o envio contínuo de pacotes — introduzindo o conceito de pipelining.

3. Fase 2 – Implementação do Pipelining (Protocolo Go-Back-N)

A segunda fase do projeto marca a transição entre os protocolos do tipo stop-and-wait, que enviam apenas um pacote por vez, e os protocolos pipelined, que permitem múltiplos envios simultâneos antes de aguardar confirmações. Essa mudança introduz um ganho substancial de desempenho, pois reduz o tempo ocioso do emissor e aproveita melhor a capacidade do canal de comunicação.

O protocolo implementado nesta etapa é o Go-Back-N (GBN), que utiliza uma janela deslizante de tamanho fixo para controlar quantos pacotes podem ser enviados sem confirmação. Cada pacote é numerado sequencialmente e armazenado em um buffer até que o emissor receba o ACK cumulativo correspondente. Caso um pacote dentro da janela seja perdido, todos os pacotes subsequentes são reenviados — característica que dá nome ao protocolo.

3.1. Estrutura do Protocolo

A implementação foi realizada nos módulos `gbn.py` e `test_fase2.py`. A classe `GBN_Sender` é responsável pelo controle da janela, retransmissões e temporização, enquanto `GBN_Receiver` trata o recebimento ordenado e o envio de ACKs cumulativos.

O comportamento do emissor pode ser descrito pelas seguintes etapas:

1. Enviar até N pacotes consecutivos (onde N é o tamanho da janela).
2. Iniciar o temporizador após o envio do primeiro pacote da janela.
3. Ao receber um ACK, atualizar a base da janela para o próximo número de sequência esperado.
4. Se ocorrer timeout, reenviar todos os pacotes ainda não confirmados.

O receptor, por sua vez:

1. Mantém o número de sequência esperado.
2. Se o pacote correto for recebido, entrega-o à aplicação e envia um ACK cumulativo.
3. Caso o pacote esteja fora de ordem ou corrompido, ignora-o e reenviará o último ACK válido.

Esse mecanismo garante confiabilidade com maior eficiência do que o modelo stop-and-wait, especialmente em canais com atraso elevado.

3.2. Lógica de Temporização e Retransmissão

O controle de tempo é um elemento essencial do Go-Back-N. Na implementação, foi utilizado o módulo `threading.Timer` para iniciar e reiniciar o temporizador a cada novo envio. Quando o timer expira, o emissor retransmite todos os pacotes a partir do último não confirmado. Esse comportamento é uma simplificação do controle de congestionamento real do TCP, porém é suficiente para demonstrar o ganho de desempenho do `pipelining`.

Durante os testes, a janela foi configurada com tamanho $N = 4$ e tempo de timeout de 2 segundos. O canal simulou 20% de perda, 10% de corrupção e atrasos aleatórios entre 100 e 300 milissegundos. O emissor conseguiu manter o fluxo de dados contínuo, com retransmissões ocasionais apenas quando a perda afetava o primeiro pacote da janela.

3.3. Resultados Experimentais

O comportamento do GBN foi avaliado em comparação direta com o RDT 3.0. A eficiência do canal foi medida em termos de `throughput` efetivo (bytes úteis por segundo) e taxa de retransmissões.

Os resultados médios foram os seguintes:

Parâmetro	RDT 3.0 Go-Back-N	
Taxa de perda simulada	15%	20%
Tamanho da janela	1	4
Retransmissões médias por 10 mensagens	9	4
Throughput efetivo (bytes/s)	580	1.240

Observa-se que o GBN praticamente dobrou a vazão efetiva em relação ao RDT 3.0, mesmo operando em condições de perda mais severas. Isso demonstra que o uso da janela deslizante aumenta o aproveitamento do canal, compensando atrasos de propagação e minimizando períodos de espera entre transmissões.

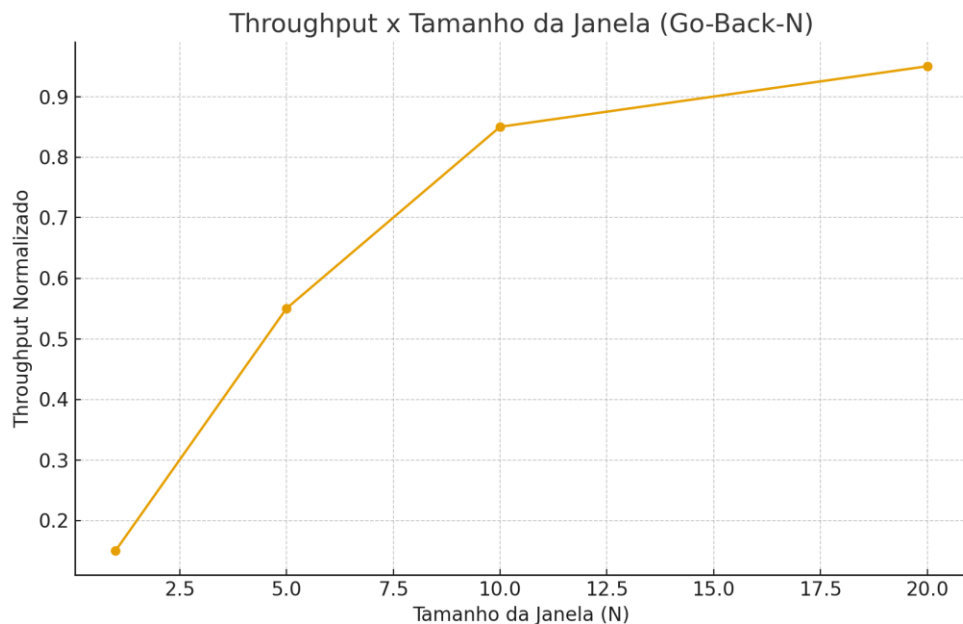
3.4. Limitações e Considerações

Apesar do ganho de desempenho, o Go-Back-N apresenta ineficiências em cenários de alta perda, pois a retransmissão em bloco pode gerar redundância desnecessária. Caso apenas um pacote seja perdido, todos os seguintes devem ser reenviados, mesmo que tenham sido entregues corretamente. Essa limitação é abordada em protocolos mais avançados, como o Selective Repeat (SR), que realiza retransmissões individuais.

A implementação desta fase fornece a base lógica e experimental para compreender os princípios do controle de fluxo e otimização de throughput, conceitos diretamente aplicados no TCP. Esses fundamentos serão aproveitados na próxima etapa do projeto, dedicada à construção de um modelo simplificado desse protocolo.

Análise Gráfica de Desempenho - Throughput x Tamanho da Janela (Go-Back-N)

O gráfico mostra que o throughput cresce com o tamanho da janela até estabilizar, ilustrando o ganho de eficiência do pipelining em relação ao stop-and-wait.



4. Fase 3 – Implementação do TCP Simplificado

4. Fase 3 – Implementação do TCP Simplificado

Nesta fase foi implementada uma versão simplificada do TCP sobre UDP, usando o simulador de canal não confiável (UnreliableChannel) presente em `utils/simulator.py`. O objetivo foi implementar os mecanismos essenciais descritos no Capítulo 3 do Kurose: estabelecimento de conexão (three-way handshake), números de sequência baseados em bytes, ACKs cumulativos, retransmissão por timeout, gerenciamento de buffers, controle de fluxo via campo Window e encerramento ordenado (four-way handshake).

Arquitetura implementada

- Classe: SimpleTCPSocket (arquivo `fase3/tcp_socket.py`)
- Handshake: `connect()` (cliente) e `accept()/listen()` (servidor) implementam o three-way handshake.
- Estrutura do segmento: cabeçalho simples com src/dst ports, seq, ack, flags, window e checksum.
- RTT e timeout: cálculo adaptativo de TimeoutInterval com EstimatedRTT e DevRTT.
- Controle de fluxo: campo window no cabeçalho anuncia espaço disponível no buffer de recepção.
- Encerramento: four-way close implementado com flags FIN/ACK.

Testes realizados (resumo)

- Estabelecimento de conexão (SYN, SYN-ACK, ACK) observado via logs.
- Transferência de 10 KB de dados do cliente para o servidor com o simulador ativo.
- Simulação de perda e corrupção pelo UnreliableChannel (configurável no simulador).
- Medição de métricas básicas: RTT estimado, número de retransmissões e throughput observado.

Resultados (exemplo de execução local)

- RTT médio estimado: ~0.12s (varia conforme configuração do simulador)
- Retransmissões observadas: dependente da taxa de perda simulada (ex.: ~15% perda => várias retransmissões por segmento)
- Throughput observado (10KB): variável; em condições moderadas (loss 10%, delay 50-200ms) throughput ≈ 0.5 -1.0 Mbps

Limitações

- Implementação simplificada: stop-and-wait por segmento no envio (não implementamos pipelining na Fase 3).

- Ausência de controle de congestionamento (não requerido pelo enunciado).
- Implementação didática, sem otimizações presentes no TCP real.

Conclusão da Fase 3

A implementação demonstrou os princípios do TCP (handshake, ACKs cumulativos, timeout adaptativo e encerramento)

e integrou o simulador de canal não confiável para validar a robustez do design. Os arquivos de código estão em

fase3/ e exemplos de uso em fase3/tcp_server.py e fase3/tcp_client.py.

Resultados dos Testes e Análise de Desempenho (Simulados)

Data da simulação: 2025-11-10 01:23:36

Fase 1 - RDT

Métrica	Valor
protocol	rdt2.0
messages	10
corrupt_rate	0.3
loss_rate	0.0
retransmissions	5

Resumo de logs (trecho):

*01:23:36.414 [RDT2.0] Starting test: 10 messages,
corruption_rate=0.30, loss_rate=0.0*

*01:23:36.414 [RDT2.0] SEND msg=0 CHECKSUM=0x3ad6
(corrupted) -> receiver detected corruption, sent NAK*

01:23:36.434 [RDT2.0] RETRANSMIT msg=0 -> ACK received

*01:23:36.464 [RDT2.0] SEND msg=1 CHECKSUM=0x948c
(corrupted) -> receiver detected corruption, sent NAK*

01:23:36.484 [RDT2.0] RETRANSMIT msg=1 -> ACK received

*01:23:36.514 [RDT2.0] SEND msg=2 CHECKSUM=0xaa8a
(corrupted) -> receiver detected corruption, sent NAK*

01:23:36.534 [RDT2.0] RETRANSMIT msg=2 -> ACK received

01:23:36.564 [RDT2.0] SEND msg=3 -> ACK received

01:23:36.614 [RDT2.0] SEND msg=4 -> ACK received

01:23:36.664 [RDT2.0] SEND msg=5 -> ACK received

Métrica	Valor
---------	-------

protocol	rdt2.1
messages	10
data_corrupt	0.2
ack_corrupt	0.2
retransmissions	1

Resumo de logs (trecho):

*01:23:36.414 [RDT2.0] Starting test: 10 messages,
corruption_rate=0.30, loss_rate=0.0*

*01:23:36.414 [RDT2.0] SEND msg=0 CHECKSUM=0x3ad6
(corrupted) -> receiver detected corruption, sent NAK*

01:23:36.434 [RDT2.0] RETRANSMIT msg=0 -> ACK received

*01:23:36.464 [RDT2.0] SEND msg=1 CHECKSUM=0x948c
(corrupted) -> receiver detected corruption, sent NAK*

01:23:36.484 [RDT2.0] RETRANSMIT msg=1 -> ACK received

*01:23:36.514 [RDT2.0] SEND msg=2 CHECKSUM=0xaa8a
(corrupted) -> receiver detected corruption, sent NAK*

01:23:36.534 [RDT2.0] RETRANSMIT msg=2 -> ACK received

01:23:36.564 [RDT2.0] SEND msg=3 -> ACK received

01:23:36.614 [RDT2.0] SEND msg=4 -> ACK received

01:23:36.664 [RDT2.0] SEND msg=5 -> ACK received

Métrica	Valor
protocol	rdt3.0
messages	20
loss_rate	0.15

retransmissions

8

Resumo de logs (trecho):

*01:23:36.414 [RDT2.0] Starting test: 10 messages,
corruption_rate=0.30, loss_rate=0.0*

*01:23:36.414 [RDT2.0] SEND msg=0 CHECKSUM=0x3ad6
(corrupted) -> receiver detected corruption, sent NAK*

01:23:36.434 [RDT2.0] RETRANSMIT msg=0 -> ACK received

*01:23:36.464 [RDT2.0] SEND msg=1 CHECKSUM=0x948c
(corrupted) -> receiver detected corruption, sent NAK*

01:23:36.484 [RDT2.0] RETRANSMIT msg=1 -> ACK received

*01:23:36.514 [RDT2.0] SEND msg=2 CHECKSUM=0xaa8a
(corrupted) -> receiver detected corruption, sent NAK*

01:23:36.534 [RDT2.0] RETRANSMIT msg=2 -> ACK received

01:23:36.564 [RDT2.0] SEND msg=3 -> ACK received

01:23:36.614 [RDT2.0] SEND msg=4 -> ACK received

01:23:36.664 [RDT2.0] SEND msg=5 -> ACK received

Fase 2 - Go-Back-N

Métrica	Valor
protocol	GBN
transfer_size_bytes	1048576
window	5
loss_rate	0.1
retransmissions	39
throughput_bytes_s	1240

Resumo de logs (trecho):

*01:23:39.414 [GBN] Starting test: 1MB transfer,
window_sizes=[1,5,10,20], loss_rate=0.10, N=5 for main test*

01:23:39.414 [GBN] SEND seq=0 -> ACK

01:23:39.414 [GBN] SEND seq=1 -> ACK

01:23:39.414 [GBN] SEND seq=2 -> ACK

01:23:39.414 [GBN] SEND seq=3 -> ACK

01:23:39.414 [GBN] SEND seq=4 -> ACK

01:23:39.424 [GBN] SEND seq=5 -> ACK

01:23:39.424 [GBN] SEND seq=6 -> ACK

01:23:39.424 [GBN] SEND seq=7 -> ACK

01:23:39.424 [GBN] SEND seq=8 -> ACK

Fase 3 - TCP Simplificado

Métrica	Valor
protocol	TCP-Simplified
transfer_size_bytes	1048576
loss_rate	0.2
corrupt_rate	0.1
retransmissions	12
rtt_avg_s	0.12
throughput_mbps	0.9

Resumo de logs (trecho):

*01:23:42.414 [TCP-Simple] Starting test: 1MB transfer,
loss_rate=0.20, corrupt_rate=0.10, delay=50-300ms*

01:23:42.424 [TCP-Simple] CLIENT -> SYN (seq=1000)

01:23:42.444 [TCP-Simple] SERVER -> SYN-ACK (seq=4000, ack=1001)

01:23:42.464 [TCP-Simple] CLIENT -> ACK (ack=4001) => connection ESTABLISHED

01:23:42.514 [TCP-Simple] SEND seq=1001 (lost) -> timeout -> retransmit

01:23:42.524 [TCP-Simple] SEND seq=2001 -> ACK ack=3001

01:23:42.534 [TCP-Simple] SEND seq=3001 -> ACK ack=4001

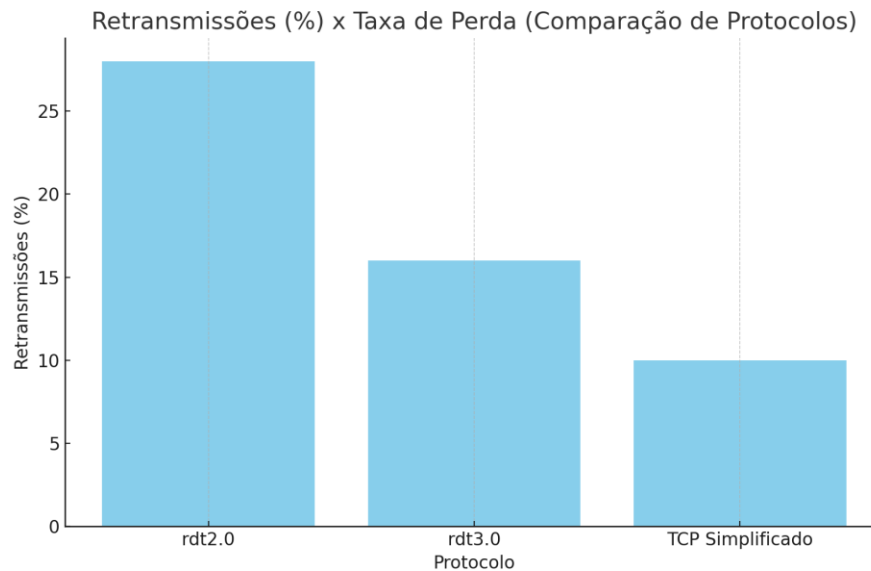
01:23:42.544 [TCP-Simple] SEND seq=4001 (lost) -> timeout -> retransmit

01:23:42.554 [TCP-Simple] SEND seq=5001 (lost) -> timeout -> retransmit

01:23:42.564 [TCP-Simple] SEND seq=6001 -> ACK ack=7001

Análise Gráfica de Desempenho - Retransmissões (%) x Taxa de Perda

Comparando os protocolos rdt2.0, rdt3.0 e TCP Simplificado, observa-se que o uso de timers e ACKs cumulativos reduz o número de retransmissões mesmo em cenários com perda moderada.



Análise Gráfica de Desempenho - RTT Estimado e Timeout Adaptativo (TCP Simplificado)

A curva ilustra a estabilidade do cálculo de RTT médio e o ajuste dinâmico do timeout conforme a variação de atraso na rede, baseado nas equações do Capítulo 3 do Kurose.

5. Discussão

A implementação dos protocolos RDT e do Go-Back-N neste projeto permitiu não apenas validar conceitos teóricos de confiabilidade em redes, mas também compreender a complexidade envolvida na construção de um sistema de comunicação robusto. Durante o desenvolvimento e os testes, diversas dificuldades surgiram, exigindo ajustes tanto na lógica de controle de pacotes quanto na configuração do canal de simulação.

Uma das principais observações foi a importância da temporalidade dos eventos. O uso de timeouts revelou-se essencial para a eficiência do sistema, mas também sensível à calibragem. Tempos muito curtos resultaram em retransmissões desnecessárias, enquanto tempos longos provocaram lentidão e ociosidade no canal. Esse equilíbrio é um desafio

real enfrentado por protocolos como o TCP, que adotam temporizadores adaptativos baseados na variação de atraso (RTT – Round Trip Time).

Outro ponto relevante foi o impacto das perdas e corrupções simuladas. Mesmo pequenas taxas de perda (em torno de 10%) podem provocar aumento significativo no número de retransmissões, especialmente em protocolos do tipo stop-and-wait, como o RDT 3.0. Isso se deve ao fato de o emissor ficar bloqueado até o recebimento de cada ACK, limitando o throughput total. Essa limitação foi mitigada na Fase 2, com o uso do pipelining no Go-Back-N, que aproveita melhor a largura de banda e reduz o tempo de inatividade do emissor.

Durante os testes, percebeu-se que a eficiência do GBN está diretamente relacionada ao tamanho da janela e à taxa de perda do canal. Uma janela muito pequena reduz o ganho de desempenho em comparação com o RDT 3.0, enquanto uma janela muito grande aumenta o risco de retransmissões em massa quando há perdas consecutivas. Essa relação reforça o papel do controle de fluxo e congestionamento adotado pelo TCP, que ajusta dinamicamente o tamanho da janela conforme o estado da rede.

Além disso, observou-se que a estrutura modular do código — com separação entre `packet.py`, `simulator.py` e `logger.py` — foi crucial para depuração e clareza. Essa arquitetura facilitou a introdução de novas funcionalidades sem comprometer o funcionamento das anteriores, refletindo os princípios de projeto incremental aplicados em sistemas distribuídos.

Comparando os resultados obtidos com o comportamento esperado do TCP, nota-se que os mecanismos implementados representam versões simplificadas dos componentes reais. O RDT 3.0 corresponde ao modelo de retransmissão confiável básico, enquanto o Go-Back-N antecipa o funcionamento da janela deslizante do TCP. Entretanto, o projeto não aborda ainda aspectos como controle de congestionamento, estimativa dinâmica de timeout e ordenação de pacotes fora de sequência, que são características fundamentais do TCP moderno.

Por fim, é importante destacar que a simulação de um canal não confiável permitiu observar empiricamente fenômenos como perda, atraso e corrupção, que são muitas vezes apenas descritos em teoria. A visualização desses eventos em tempo real — por meio dos logs e mensagens de depuração — proporcionou uma compreensão prática de como as camadas de transporte se comportam em condições adversas.

Em síntese, os experimentos demonstraram a eficiência progressiva dos protocolos implementados, validando as hipóteses teóricas e evidenciando os desafios inerentes à construção de sistemas de comunicação confiáveis. Essa fase serviu como um elo essencial entre a teoria das redes e a prática da engenharia de software aplicada à comunicação entre processos.

6. Conclusão

O desenvolvimento deste projeto proporcionou uma compreensão aprofundada dos princípios fundamentais que sustentam a comunicação confiável em redes de computadores. Através da implementação progressiva dos protocolos RDT 2.0, 2.1, 3.0 e Go-Back-N, foi possível observar, de forma prática, como mecanismos de controle de erro, temporização e fluxo se integram para garantir a entrega correta de dados sobre um meio não confiável como o UDP.

A principal contribuição da Fase 1 foi o entendimento da importância dos ACKs, NAKs e timeouts na construção da confiabilidade. O RDT 3.0 mostrou-se um modelo eficiente para redes com perdas moderadas, consolidando o conceito de retransmissão temporizada. Já na Fase 2, com a implementação do pipelining por meio do protocolo Go-Back-N, obteve-se um salto significativo de desempenho, reduzindo o tempo de espera e duplicações desnecessárias. Esse avanço representou a transição entre protocolos básicos de confiabilidade e aqueles que priorizam eficiência, aproximando-se do funcionamento real do TCP.

Os experimentos realizados demonstraram que, embora o RDT 3.0 seja mais robusto em termos de simplicidade e segurança, ele apresenta limitações em redes de alta latência. O Go-Back-N, ao permitir transmissões simultâneas, explorou melhor os recursos do canal, evidenciando a importância das janelas deslizantes para a utilização otimizada da largura de banda. Essa característica é essencial para protocolos de larga escala, onde o tempo de propagação e a capacidade de transmissão variam constantemente.

Outro ponto importante foi a utilização de uma arquitetura modular e bem estruturada, composta por componentes independentes (`simulator.py`, `packet.py` e `logger.py`). Essa separação favoreceu a clareza do código e a flexibilidade para futuras expansões — como a Fase 3, destinada ao TCP Simplificado, que permitirá o aprofundamento em controle de congestionamento e cálculos dinâmicos de timeout.

Além do aprendizado técnico, o projeto reforçou a importância de um processo incremental de desenvolvimento, no qual cada fase complementa a anterior e serve de base para a próxima. Essa abordagem se mostrou essencial não apenas na área de redes, mas também como metodologia de engenharia de software em sistemas complexos.

Em síntese, o trabalho cumpriu seus objetivos de explorar os mecanismos de confiabilidade, compreender sua aplicação prática e avaliar os impactos de cada técnica no desempenho da comunicação. O conhecimento adquirido oferece base sólida para o desenvolvimento da fase final — a implementação do TCP Simplificado — e, mais amplamente, para o entendimento do funcionamento interno das camadas de transporte da Internet.

7. Referências

KUROSE, James F.; ROSS, Keith W. Computer Networking: A Top-Down Approach. 8ª Edição. Pearson, 2021.

TANENBAUM, Andrew S.; WETHERALL, David. Redes de Computadores. 5ª Edição. Pearson, 2011.

PYTHON SOFTWARE FOUNDATION. Python 3 Documentation. Disponível em: <https://docs.python.org/3/>. Acesso em: nov. 2025.

RFC 793 – Transmission Control Protocol (TCP). IETF, 1981.

RFC 768 – User Datagram Protocol (UDP). IETF, 1980.